Report 2 - Faisal Farzana, Keith Hussain, Genaro Salazar Ruiz

Question 1

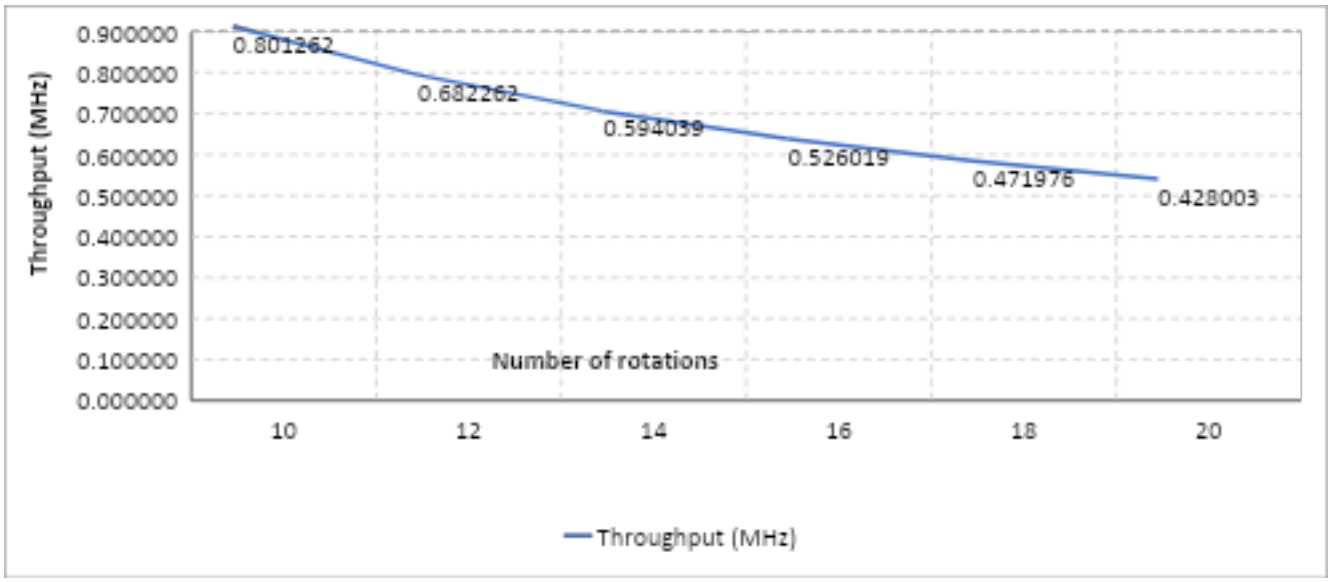
1a:

Table

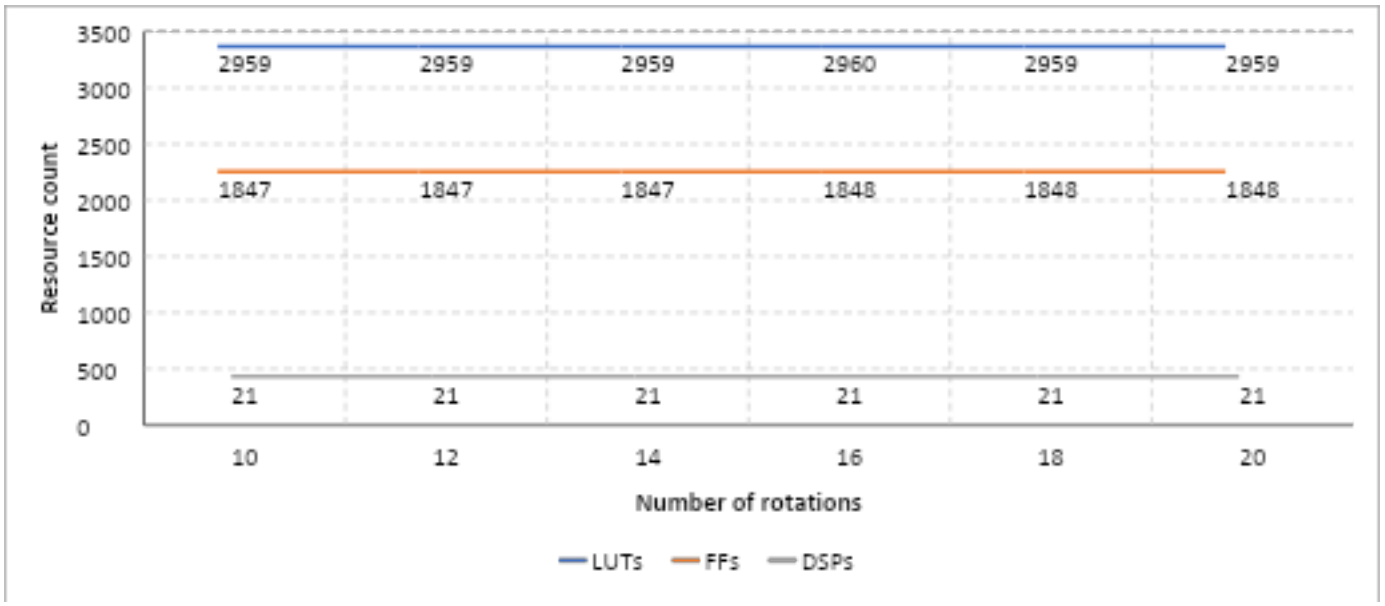
Rotations	Estimated Clock (ns)	Max Latency (cycles)	Max Interval (cycles)	Throughput (MHz)	LUTs	FFs	DSPs	BRAM	URAM	RMSE (r)	RMSE (theta)
10	7.256	171	172	0.801262	2959	1847	21	0	0	1.267448283E-06	1.373913954E-03
12	7.256	201	202	0.682262	2959	1847	21	0	0	6.273187100E-08	2.748398401E-04
14	7.256	231	232	0.594039	2959	1847	21	0	0	1.087940120E-07	8.103813889E-05
16	7.256	261	262	0.526019	2960	1848	21	0	0	1.087940120E-07	1.683781193E-05
18	7.256	291	292	0.471976	2959	1848	21	0	0	1.087940120E-07	4.681508926E-06
20	7.256	321	322	0.428003	2959	1848	21	0	0	1.087940120E-07	6.017906460E-07

1b:

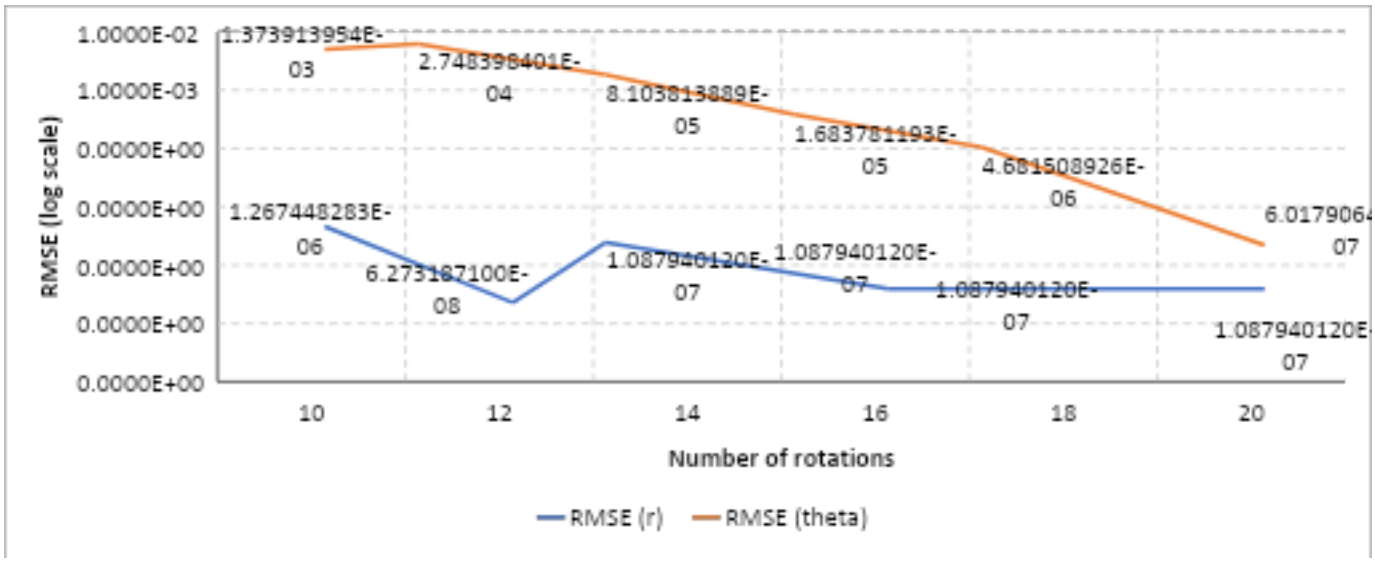
Throughput Plot



Resource Usage Plot



RMSE Plot



1c:

For the radius RMSE it is at its lowest at 12 rotations, it then slightly increases at 14 rotations and then stops noticeably changing in accuracy. For theta RMSE it continues to drop as we increase in rotations. The logarithmic scale helps to visually display the RMSE for these small values.

<u>Question 2</u>

2a:

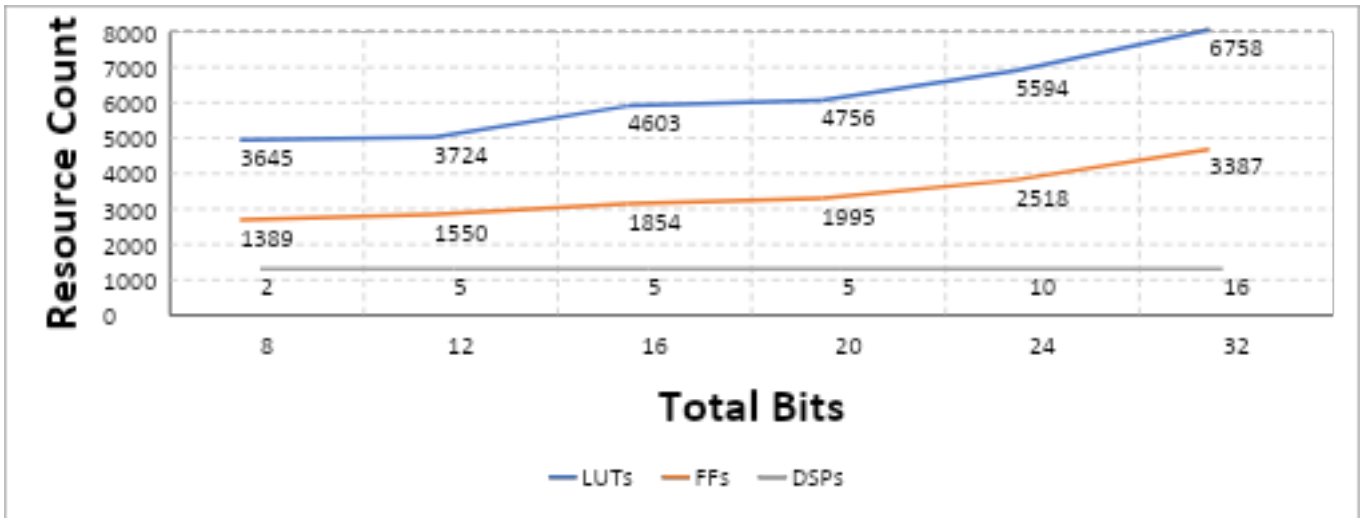
The minimum amount of integer bits for angles[ ] is 1 integer bit, for r, Kvalues[ ], x, y, x_shift, y_shift the minimum is 2 integer bits, and for theta, current_x, current_y, current_theta the minimum is 3 integer bits.

2b:

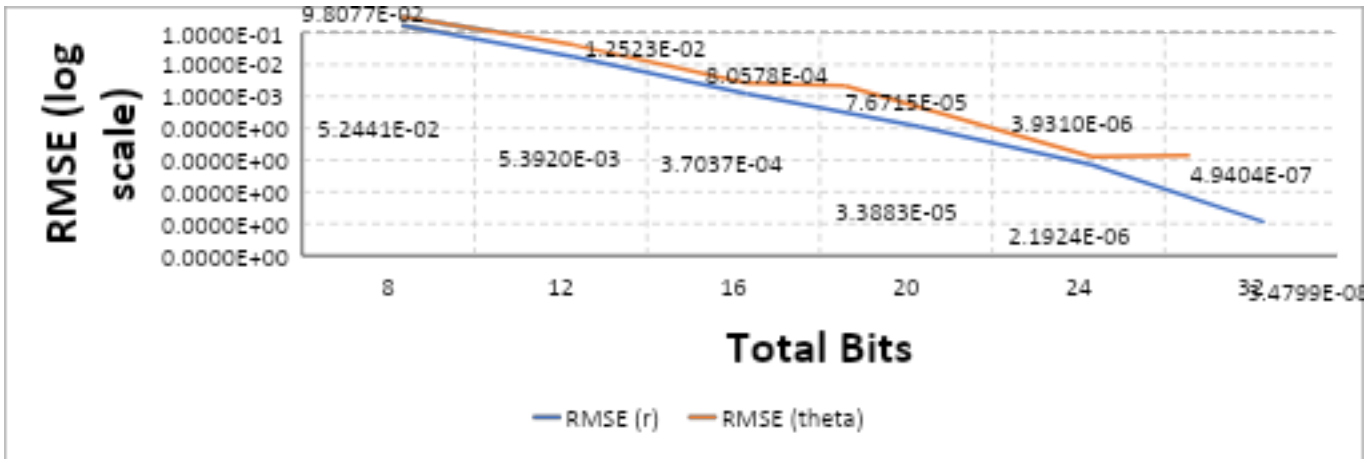
Table

Total Bits	Estimated Clock (ns)	Max Latency (cycles)	Max Interval (cycles)	Throughput (MHz)	LUTs	FFs	DSPs	BRAM	URAM	RMSE (r)	RMSE (theta)
8	7.256	57	58	2.376155	3645	1389	2	0	0	5.2441E-02	9.8077E-02
12	7.300	58	59	2.321802	3724	1550	5	0	0	5.3920E-03	1.2523E-02
16	7.256	59	60	2.296950	4603	1854	5	0	0	3.7037E-04	8.0578E-04
20	7.256	60	61	2.259295	4756	1995	5	0	0	3.3883E-05	7.6715E-05
24	7.256	60	61	2.259295	5594	2518	10	0	0	2.1924E-06	3.9310E-06
32	7.256	82	83	1.660446	6758	3387	16	0	0	3.4799E-08	4.9404E-07

Resource Usage Plot



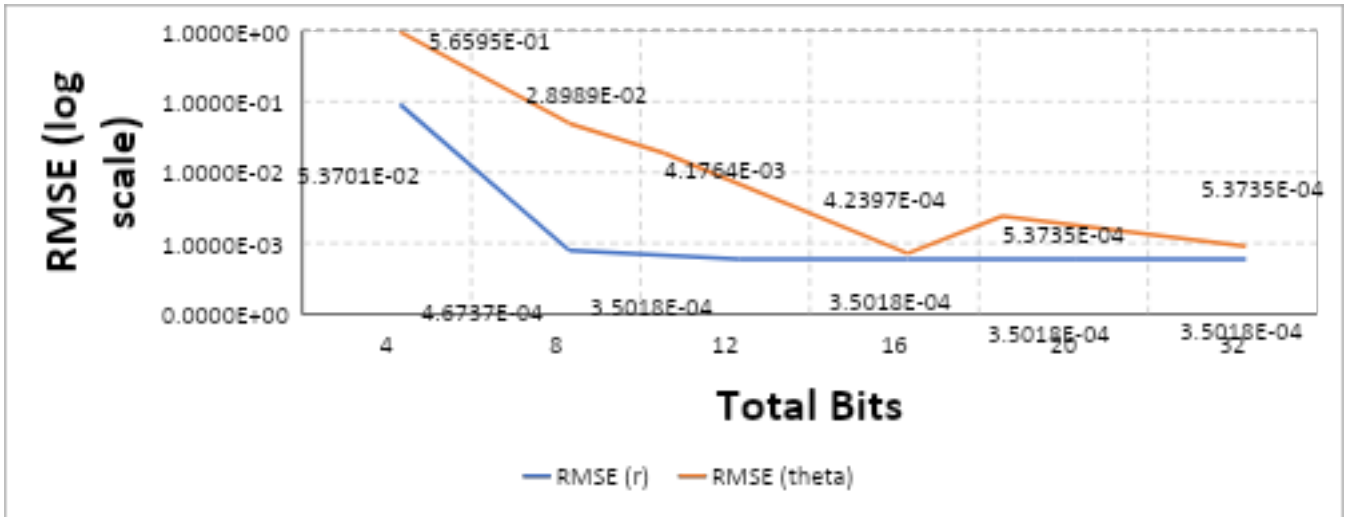
RMSE Plot



2c:

Table

Total Bits	Estimated Clock (ns)	Max Latency (cycles)	Max Interval (cycles)	Throughput (MHz)	LUTs	FFs	DSPs	BRAM	URAM	RMSE (r)	RMSE (theta)
4	7.256	59	60	2.296950	4538	1772	3	0	0	5.3701E-02	5.6595E-01
8	7.256	59	60	2.296950	4556	1784	5	0	0	4.6737E-04	2.8989E-02
12	7.256	59	60	2.296950	4562	1800	5	0	0	3.5018E-04	4.1764E-03
16	7.256	59	60	2.296950	4573	1816	5	0	0	3.5018E-04	4.2397E-04
20	7.256	59	60	2.296950	4600	1846	5	0	0	3.5018E-04	5.3735E-04
32	7.256	79	80	1.722712	4741	2133	7	0	0	3.5018E-04	5.3735E-04



Question 3

3a:

Q3 Shift/Addition Implementation Table

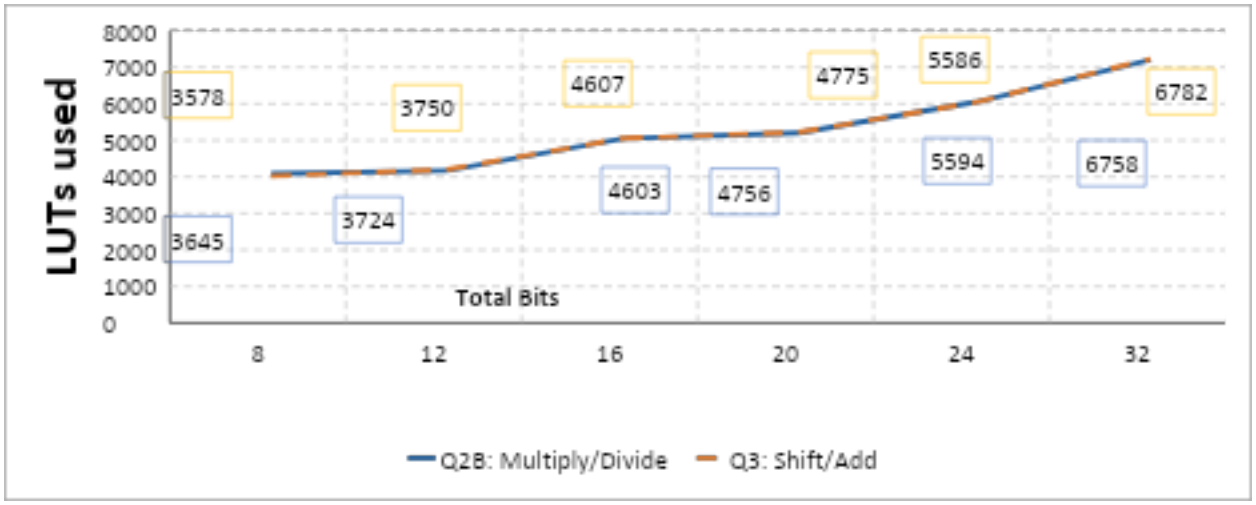
Total Bits	Estimated Clock (ns)	Max Latency (cycles)	Max Interval (cycles)	Throughput (MHz)	LUTs	FFs	DSPs	BRAM_18K	URAM	RMSE (r)	RMSE (theta)
8	7.256	58	59	2.335881	3578	1357	2	0	0	2.0748E-01	9.7920E-02
12	7.256	58	59	2.335881	3750	1466	3	0	0	1.0438E-02	1.0189E-02
16	7.256	58	59	2.335881	4607	1759	3	0	0	5.4744E-04	8.0394E-04
20	7.256	59	60	2.296950	4775	1876	3	0	0	1.3109E-05	6.8659E-05
24	7.256	59	60	2.296950	5586	2374	6	0	0	7.1208E-07	3.9124E-06
32	7.256	61	62	2.222855	6782	2866	8	0	0	3.4799E-08	6.2870E-07

Q2B Multiplication/Division Implementation Table

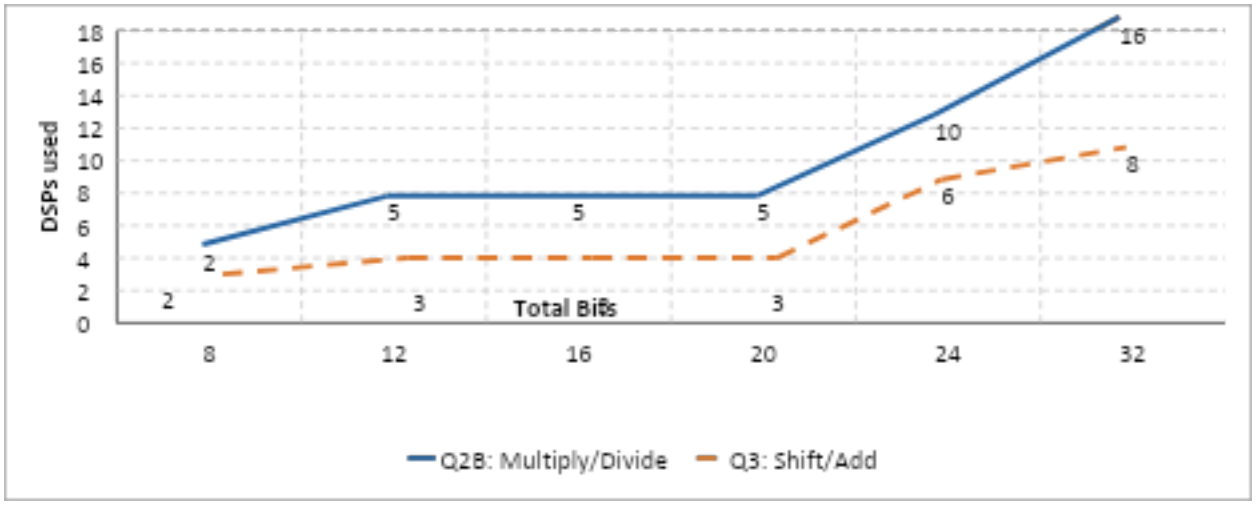
Total bits	Estimated Clock (ns)	Max Latency (cycles)	Max Interval (cycles)	Throughput (MHz)	LUTs	FFs	DSPs	BRAM	URAM	RMSE (r)	RMSE (theta)
8	7.256	57	58	2.376155	3645	1389	2	0	0	5.2441E-02	9.8077E-02
12	7.300	58	59	2.321802	3724	1550	5	0	0	5.3920E-03	1.2523E-02
16	7.256	59	60	2.296950	4603	1854	5	0	0	3.7037E-04	8.0578E-04
20	7.256	60	61	2.259295	4756	1995	5	0	0	3.3883E-05	7.6715E-05
24	7.256	60	61	2.259295	5594	2518	10	0	0	2.1924E-06	3.9310E-06
32	7.256	82	83	1.660446	6758	3387	16	0	0	3.4799E-08	4.9404E-07

3b:

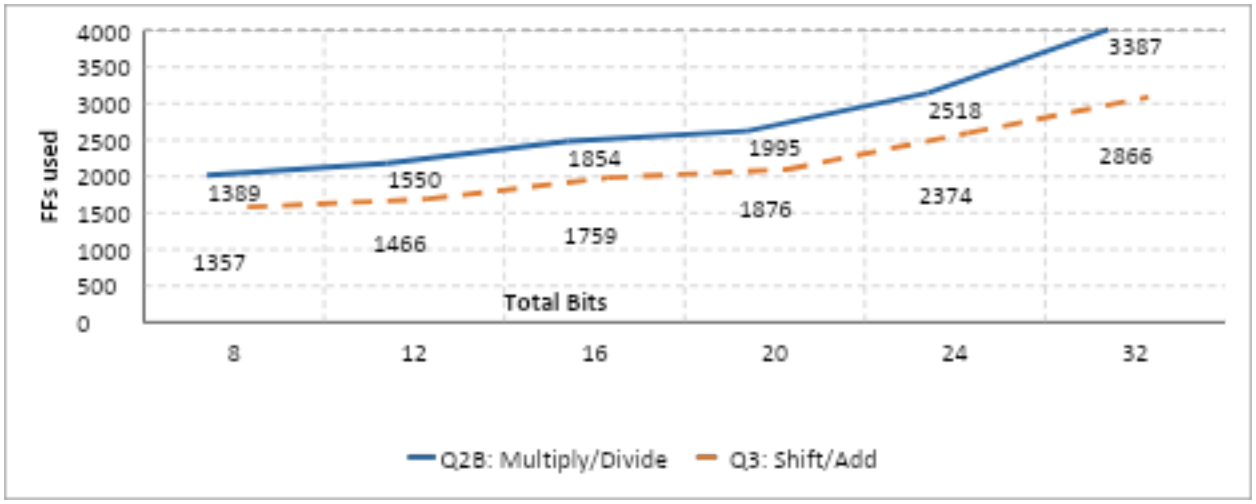
LUT Plot



DSP Plot



FF Plot



When you use shift/add operations instead of multiplication/division we saw consistent improvement in many of the resource usage parameters. In FFs and DSPs we saw considerable improvement in resource usage, but LUTs usage is about the same.

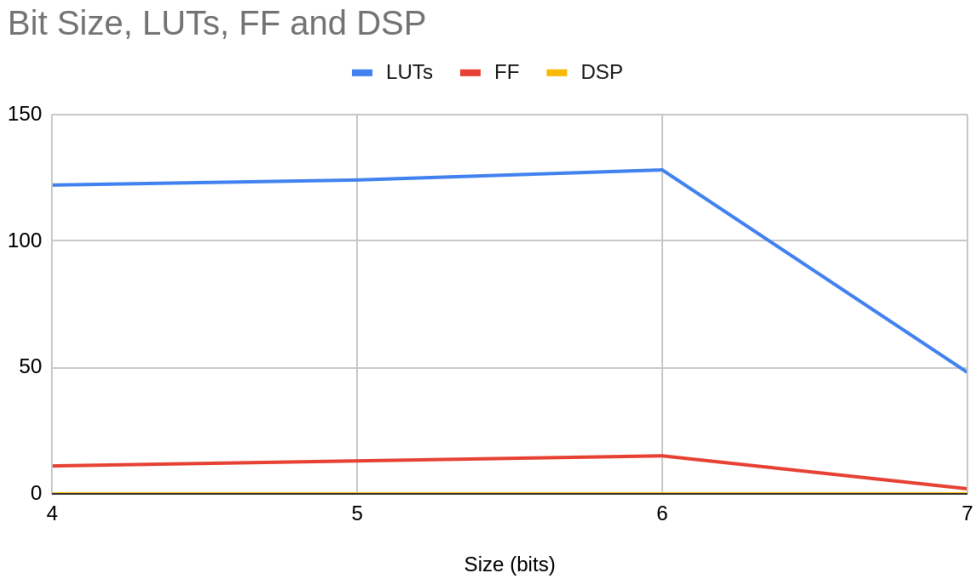
Question 4

a. The LUT size is determined by 2^{2W}, where W is the total amount of bits for the fixed point representation for the input data.

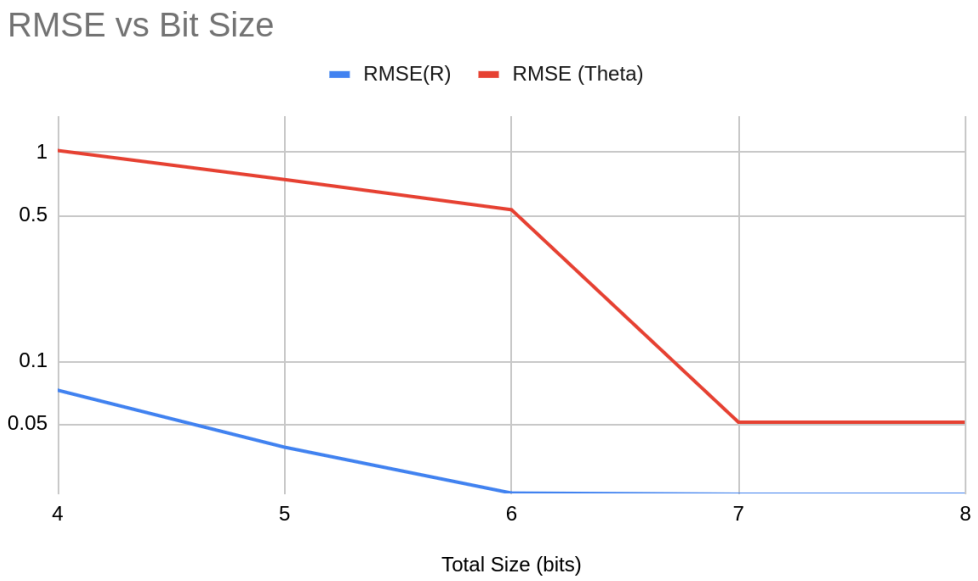
b.

Bit Size	LUTs	FF	DSP
4	122	11	0
5	124	13	0
6	128	15	0
7	48	2	0
8	N/A	N/A	N/A

c. *Synthesis of 8 bit input size was failing likely due to memory constraints on the server and was not achieved as of the writing of this report.*



d.



e. The main advantage of the CORDIC implementation is that synthesis doesn't take nearly as long as the LUT-based implementation. In addition, the memory usage of the CORDIC implementation is nowhere near as high as LUT-based. However, the main disadvantage is that because the values are computed in real time, the performance won't be as high as LUT-based. In addition, while the memory usage is low, CORDIC requires much more

of all of the other resources (DSPs, FFs, and LUTs), which can contribute to a higher cost to build.

Demo Link

https://youtu.be/E4LwYVhYpdQ

GitHub Repository

https://github.com/faisalfarzana/WES-237C-CORDIC-Lab-2/tree/main

Phase Detector:

–Note:

For FIR_BASELINE: Added cordiccart2pol.cpp, phasedetector.cpp, phasedetector.h, as these files were needed to make report.

Question 1: What is the throughput of your Phase Detector? How does that relate to the individual components (FIR, CORDIC, etc.)? How can you make it better?

1. Hardware Results Summary Table

Design Architecture	Latency (Cycles)	Initiation Interval (II)	Achieved Period	LUTs	FFs	DSPs	BRAM_18K
Baseline	257 – 510	258 – 511	7.256 ns	5,696 (10%)	3,613 (3%)	31 (14%)	12 (4%)
Optimized 1 (Loop Pipelining)	194 – 383	195 – 384	12.958 ns	5,942 (11%)	3,612 (3%)	31 (14%)	12 (4%)

Optimized 2 (Top-Level II=1)	417	1	7.256 ns	101,444 (190%)	75,820 (71%)	597 (271%)	0 (0%)

Part 1: What is the throughput of your Phase Detector?

- **Baseline:** The initiation interval varies between 258 and 511 clock cycles per sample, depending on control-branch paths in the CORDIC pre-rotation. At a 100 MHz target clock, this corresponds to a sample rate of 195.7 kSPS up to 387.6 kSPS.
- **Optimized 1:** Loop-level pipelining reduces the initiation interval to 195-384 clock cycles per sample, which increases throughput by roughly 25%.
- **Optimized 2:** Top-level pipelining achieves the theoretical maximum throughput: 1 sample per clock cycle with a constant latency of 417 cycles. At 100 MHz, this delivers a continuous throughput of 100 MSPS.

Part 2: How does that relate to the individual components (FIR, CORDIC, etc.)?

- **Baseline Architecture:** The components execute completely in series.
  - The complex matched filter consumes 247 clock cycles executing sequential shift-register updates and mult-acc operations across the four sub-filters.
  - The CORDIC engine uses 242 clock cycles running its 16 vectoring iterations.
  - Total initiation interval is bounded by the sum of both sequential stages ($II_{total} \approx Latency_{FIR} + Latency_{CORDIC}$).
- **Optimized 1:** Pipelining the arithmetic inner loops reduces the FIR latency from 247 down to 184 cycles and CORDIC latency from 242 down to 178 cycles, reducing overall II proportionally.
- O**ptimized 2:** Fully unrolling and pipelining the datapath relieves latency from throughput. The intermediate registers hold intermediate states, allowing every stage of the 4 FIR sub-filters and the 16 CORDIC iterations to compute concurrently on each sequential sample stream without a bottleneck.

Part 3: How can you make it better?

- **Hardware Area Optimization:** While Optimized 2 achieves $II = 1$, it overutilizes the target Zynq FPGA. You can see it requires 597 DSPs vs. 220 available, and 101,444 LUTs vs. 53,200 available.

- **Targeted II Relaxation:** Constraining the top-level pipeline to $II = 2$ or $II = 4$ allows DSP multipliers and adders to be time-multiplexed between the FIR sub-branches, which optimizes the hardware footprint by more than 50%.
- **Fixed-Point Conversion:** Migrating from single-precision 32-bit floating-point to custom fixed-point arithmetic of 16 bit or lower, depending on input/output.